

SLIDE 1 — What is a Multi-Dimensional Array?

Definition:

A **multi-dimensional array** is an array that stores data in **more than one dimension**, such as rows and columns.

Simple idea

Rows → **Columns** → **Values**

Example:

```
int matrix[2][3] = {  
    {10, 20, 30},  
    {40, 50, 60}  
};
```

Explanation:

It contains **2 rows** and **3 columns**.

	Col 0	Col 1	Col 2
Row 0	10	20	30
Row 1	40	50	60

 **Remember: 2D Array = Rows + Columns**

SLIDE 2 — How Does a 2D Array Work?

Structure

```
dataType arrayName[rows][columns];
```

Example

```
int numbers[2][3];
```

This creates:

2 rows × 3 columns = 6 elements

```
[][][]  
[][][]
```

★ Remember

● First index → **Row**

● Second index → **Column**

`numbers[1][2]`

means:

Row 1 → Column 2

SLIDE 3 — Accessing Elements

A 2D array uses **two indexes** to access an element.

Example

```
int matrix[2][3] = {  
    {10, 20, 30},  
    {40, 50, 60}  
};
```

```
printf("%d", matrix[1][2]);
```

Explanation:

	0	1	2	
0	10	20	30	
1	40	50	60	

↑
`matrix[1][2]`

Output:

60

 Remember: **[row][column]** → Find the element

SLIDE 4 — Using Loops with 2D Arrays

Nested loops are commonly used to access all elements.

Example

```
int matrix[2][3] = {
    {10, 20, 30},
    {40, 50, 60}
};

for (int i = 0; i < 2; i++) {
    for (int j = 0; j < 3; j++) {
        printf("%d ", matrix[i][j]);
    }
}
```

Explanation:

i → Row

j → Column

The loops visit:

matrix[0][0] → 10
matrix[0][1] → 20
matrix[0][2] → 30
matrix[1][0] → 40
matrix[1][1] → 50
matrix[1][2] → 60

Output:

10 20 30
40 50 60

 Basic Pattern:

Outer Loop → Rows

Inner Loop → Columns

SLIDE 5 — RECAP

2D Array = Rows + Columns! 

Flow

2D ARRAY



CHOOSE ROW



CHOOSE COLUMN



ACCESS ELEMENT

5 Things to Remember

- 1. A 2D array stores data in rows and columns**
- 2. It uses two indexes**
- 3. First index represents the row**
- 4. Second index represents the column**
- 5. Nested loops can access all elements**

Final Example

```
int matrix[2][2] = {  
    {10, 20},  
    {30, 40}  
};
```

```
printf("%d", matrix[1][0]);
```

Row 1 → Column 0 → 30

Output:

⚡ Memory Trick

1st Index → ROW ↑

2nd Index → COLUMN ↔

`array[row][column]` → FIND IT! 🔍

QUESTIONS

1. What is a multi-dimensional array?

- A) An array with only one value
- B) An array with multiple dimensions
- C) A single variable
- D) A loop

💡 **Hint:** Think about rows and columns.

📖 **Glossary:** Multi-dimensional → Having more than one dimension.

✅ **Answer:** B

2. How many elements are in this array?

```
int matrix[2][3];
```

- A) 2
- B) 3
- C) 5
- D) 6

💡 **Hint:** Multiply rows × columns.

📖 **Glossary:** Dimension → A direction or level used to organize data.

✅ **Answer:** D) 6

3. What is the output?

```
int matrix[2][3] = {  
    {10, 20, 30},  
    {40, 50, 60}  
};
```

```
printf("%d", matrix[0][2]);
```

 **Hint:** Go to row 0, then column 2.

 **Glossary:** Column → A vertical group of elements.

 **Answer:** 30

4. Fill in the blank

Complete the code to access the element at **row 1, column 2**:

```
int matrix[2][3] = {  
    {10, 20, 30},  
    {40, 50, 60}  
};
```

```
printf("%d", matrix[___][___]);
```

 **Hint:** First write the row, then the column.

 **Glossary:** Row → A horizontal group of elements.

 **Answer:** 1][2

5. In `matrix[i][j]`, what does **i** usually represent?

- A) Column
- B) Row
- C) Value
- D) Size

 **Hint:** The first index represents the row.

 **Glossary:** Index → Position used to access an element.

✓ Answer: B) Row

6. What is commonly used to process all elements of a 2D array?

- A) Nested loops
- B) Only `if`
- C) Only `printf()`
- D) Variables only

💡 **Hint:** One loop can handle rows and another can handle columns.

📖 **Glossary:** Nested loop → A loop placed inside another loop.

✓ Answer: A) Nested loops

7. Arrange the steps in the correct order:

- A. Access the element
- B. Choose the row
- C. Choose the column
- D. Use `array[row][column]`

💡 **Hint:** First select the row, then the column.

📖 **Glossary:** Access → Retrieve a particular element.

✓ Answer: B → C → D → A